

Ubisoft Next 2026 (Metal Invasion) Documentation

January 2026

Contents

Overview of Game + Controls	1
Overview of systems + relevant implementation details	1
Game initialization and run flow	3
References	4

Overview of Game + Controls

Metal Invasion is a survival tower defense RTS game where you control and build your army versus hordes of enemy invaders. At the start of each round you gather resources (crystals) to build up your army, after a certain amount of time the enemies will start spawning. It is your job to defend the base until the very end.

The controls are simple:

1. Left click a unit to select and click again to guide them across the map
2. Space bar to group to units of your own together
3. Click on your base to bring up Army Units or Obstacles to buy

These are the units available to the players

1. Soldiers units: Basic units that will attack any enemies nearby
2. Support units: support units help build your crystals to buy more units with park them next to a crystal and they will automatically start mining
3. Tanks units: Tanks are powerful units that can launch explosive projectiles. But the explosion can affect friendly units too so they require careful placement.
4. Obstacles: Obstacles help block enemy path to funnel enemy units into a chokehold

Overview of systems + relevant implementation details

The major feature systems in this game are:

1. ECS System
2. 3D Renderer
3. Physics System
4. AI System
5. UI System

ECS System

The Entity Component System (ECS) is the core backbone of all systems within this game. It organizes game logic by separating data (**Components**) from behavior (**Visitors/Systems**) and grouping them in a centralized controller (**ECSManager**). An Entity is represented by a unique integer ID, and each Entity can be dynamically associated with various Components that store specific bits of data—like position or velocity. The ECSManager then keeps track of these Entity-Component relationships through Signatures, which are bitsets indicating which Components an Entity contains. By registering different Visitor types, the ECSManager can run logic that targets only Entities having the required combination of Components.

Under the hood, the ComponentManager manages the ComponentBuffer classes, which store data contiguously in arrays for efficient iteration and memory caching. When you add a Component to an Entity, its data is inserted into the appropriate ComponentBuffer, and a signature bit is flipped to indicate that the Entity now has that Component. Conversely, removing a Component from an Entity updates its signature and reorganizes the ComponentBuffer to keep storage packed.

The VisitorManager handles Visitors, which are objects that gather Entities meeting the requirements for each System. These internal sets of Entities are updated whenever signatures change or Entities are destroyed.

Additionally, the ECS system supports Resources, which are global singletons that can persist for the entire game session. They are tracked by the ECSManager in a dedicated list and identified by type information, preventing duplication. Examples of resources in this game include Cameras, VertexBuffer, and IndexBuffer.

3D Renderer

As part of my preparation for the contest, I wanted a highly modular 3D renderer capable of accommodating a variety of art styles. As such, the renderer likely took the longest to implement, as it supports a myriad of features:

1. Fully programmable pipeline (vertex and fragment shading)
2. Z-depth buffer
3. Orthographic or perspective projection
4. Shadow mapping
5. Line rendering

In order to achieve a fully programmable pipeline using only the provided API, the graphics code outputs to a pixel buffer and calls `App::DrawLine` on every pixel (drawing a tiny line). Ideally, rendering to an OpenGL texture with a single draw call would be more performant, or offloading the entire rasterization to a GPU. Nonetheless, the renderer is still fairly performant, running at about 30 FPS on a Ryzen 5600X CPU. For machines with weaker CPUs, I implemented less performance-intensive options, such as line rendering or reducing the shadow map size.

Two crucial optimizations keep this software rasterizer viable:

1. **Concurrency:** After clipping, the rasterizer uses a tile-based approach to assign triangles to discrete screen regions. Each tile is rasterized in parallel, processing small pixel blocks within triangle bounding boxes.
2. **SIMD:** The renderer uses AVX256 instructions to shade batches of up to 8 pixels simultaneously.

To see the impact of these optimizations, comment out `#define PARALLEL` in `Concurrent.h` and `#define SIMD_AVAILABLE` in `SIMD.h`.

Physics System

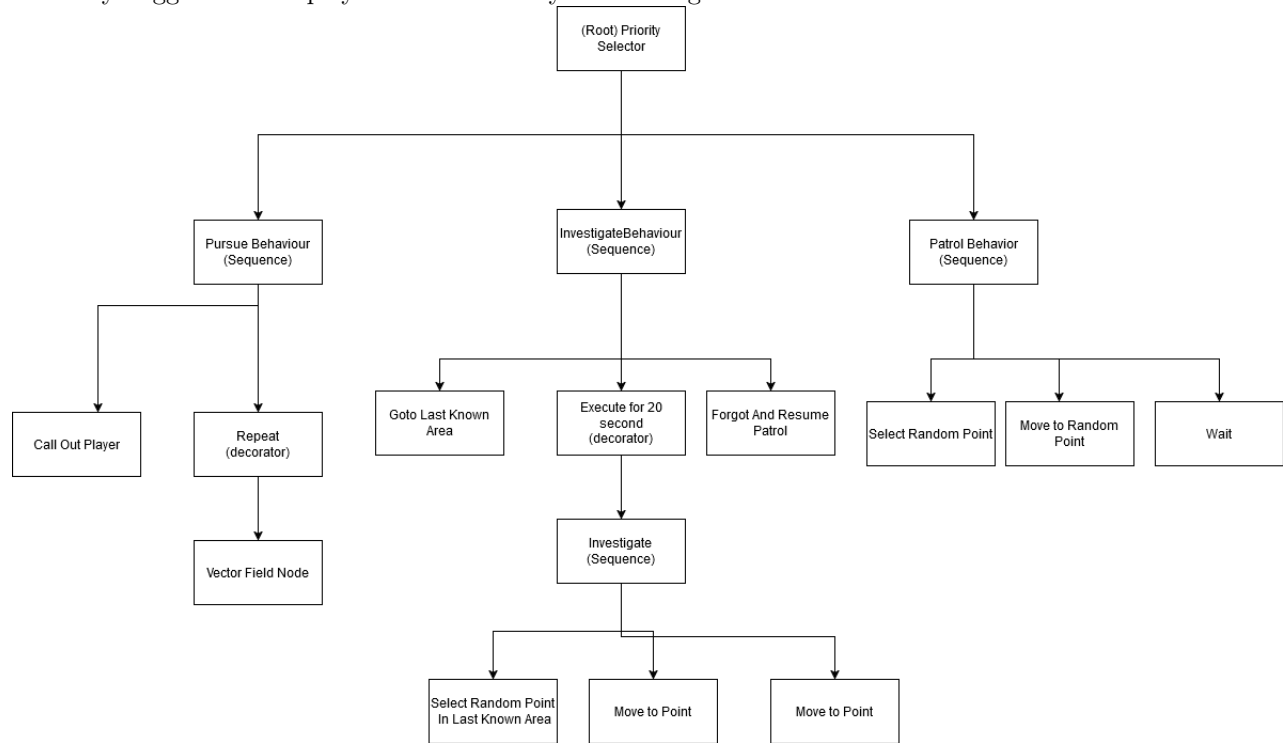
The Physics System is a basic 2D physics engine that supports box and circle collision objects. Custom collider callbacks are also supported. Currently, the Physics System implements a broad-phase collision check using AABB bounding boxes. In the interest of time, more advanced optimizations such as quad trees have been omitted. Performance and accuracy can be tweaked via the `StepTime` or `STEP_ITERATION` variables in `PhysicsSystem.cpp` and `PhysicsSystem.h`.

AI System

This AI system is built around a Behavior Tree architecture with a Blackboard for centralized pathfinding data. The AISystem processes all Entities that have a BehaviorTree, running their node hierarchies from a global database. Some nodes loop or sequence child nodes (e.g., RepeatNode, SequenceNode, SelectorNode), while others, such as VectorFieldNode, pull from the Blackboard's 2D grid to move the entity toward the player. This grid is updated using a Dijkstra-based algorithm that computes each cell's `NextX` and `NextY`, directing an entity one step closer to the target.

Whenever new obstacles (through the Obstacle component) appear or disappear, the BlackboardSystem modifies the VectorField and recalculates paths so AI entities can automatically reroute. By storing distances and next-move pointers in MapLoc structs, all AI agents can share the same grid, simplifying pathfinding.

The Behavior Tree allows for fairly complex AI behavior through a modular set of nodes. For example, below is an example of a AI which mimics behaviors from stealth games like metal gear solid. The AI can pursue the player, investigate, or patrol. The root of the tree is a priority selector node, which prioritizes children in the order they are added: Pursuit, Investigate, and Patrol. Every tick, the tree checks conditions to see if each subtree can run. For instance, the Pursuit Behavior is only triggered if the player is in the enemy's line of sight.

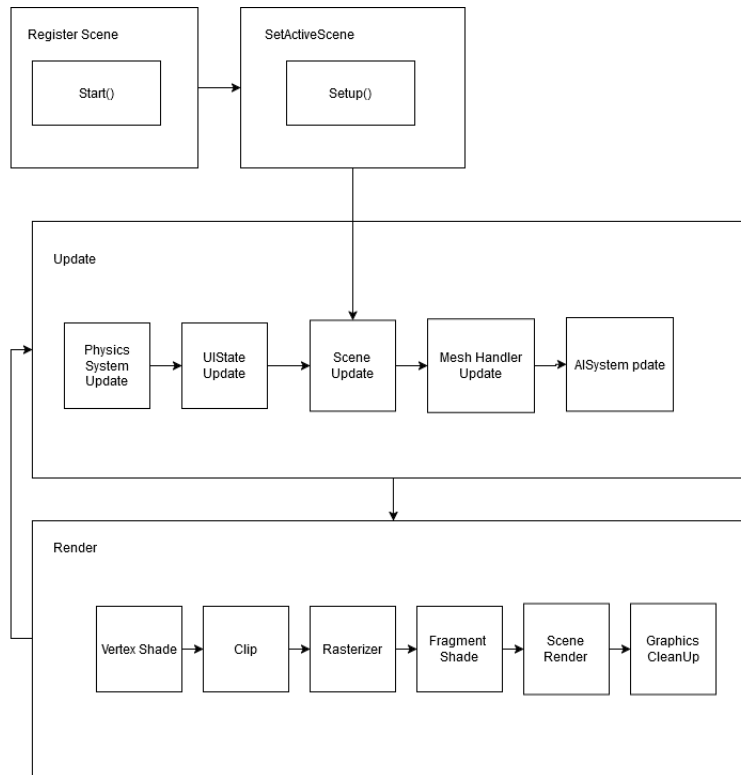


UI System

A basic immediate-mode UI library was implemented as part of the game engine. The UI system renders and updates widgets each frame using a shared UIState. It checks user interactions by comparing the mouse position against each widget's bounding box to determine whether a widget is “hot” (hovered) or “active” (clicked).

Game initialization and run flow

The two top-level objects within the game are the **ECSManager** and the **GameManager**. The ECSManager is a global object that any system can access to interact with the Entity Component System. The GameManager orchestrates systems that gameplay programmers use, such as physics, scene management, and rendering. The GameManager is composed of Scenes, each responsible for the individual gameplay logic of a level. The diagram below outlines the general flow of the game engine:



1. **RegisterScene**: Called once at startup to add a scene into the GameManager.
2. **SetActiveScene**: Switches the currently active scene. This can be called repeatedly during the game loop.
3. **Physics System Update**: Steps the physics simulation, synchronizes all rigidbodies with their corresponding transforms, and calls any relevant user-defined collision callbacks.
4. **UIState Update**: Updates the global UIState object, which gameplay programmers use to detect mouse position and clicks.
5. **Scene Update**: Handles any arbitrary gameplay logic.
6. **Mesh Handler Update**: Synchronizes Mesh transforms with the rendering system. It tracks any Meshes that are added or removed from the game and updates the VertexBuffer accordingly. It is also responsible for linking Meshes with their Shaders.
7. **AISystem/BlackBoardSystem Update**: The BlackboardSystem calculates the vector field pointing to the player using a Dijkstra-based algorithm. The AISystem runs through all registered Behavior Trees, updating enemy behavior.
8. **Vertex Shade**: Transforms vertices to clip space.
9. **Clip**: Performs triangle clipping using the Cohen–Sutherland algorithm.
10. **Rasterizer**: Uses a tiled assignment strategy, followed by multi-threaded tile-based rasterization.
11. **Fragment Shade**: Calls the fragment shader, which writes to the pixel buffer and displays to the screen.
12. **Graphics Cleanup**: Releases resources used in the current frame’s pipeline, preparing for the next frame.

References

These are the resources I used and referenced to create this submission:

1. [Larrabee Rasterization](#)
This article describes how to perform multi-threaded rasterization. I referenced it a lot when building my tile-based renderer.

2. [Fast CPU Rasterizer](#)
This GitHub repository contains a lot of good SIMD examples applied to rasterization and taught me a great deal about speeding up code with SIMD instructions.
3. [ECS System](#)
This tutorial guided me when building the underlying ECS system for my game. I have since heavily reworked it for easier usage and improved performance.
4. [Code-It Yourself 3D Graphics](#)
I originally built a renderer based on this YouTube tutorial series before rewriting it to be multi-threaded.
5. [How to Create a Custom 2D Physics Engine](#)
A tutorial for building a 2D physics engine. I did not reference it much, but it was useful for structuring my physics engine.
6. [Separating Axis Theorem](#)
An explanation of the Separating Axis Theorem, which was crucial for determining collision normals and penetration depth.
7. [Contact Points](#)
This was the best tutorial I found on how to locate contact points after collision.
8. [Let's Make a Physics Engine](#)
An excellent YouTube tutorial that clearly explains the math behind physics engines. I used it frequently when implementing rotational dynamics in my physics subsystem.